

Table of Contents

1. Introduction.....	2
1.1 Purpose.....	2
1.2 Scope.....	2-3
1.3 Definitions, acronyms, and abbreviations.....	3
1.4 References.....	3-4
1.5 Overview.....	4
2. Overall description.....	4
2.1 Product perspective.....	4-6
2.2 Product functions.....	7
2.3 User characteristics.....	7
2.4 Constraints.....	7
2.5 Assumptions and dependencies.....	8
3. Specific requirements.....	8-14
Appendixes.....	15-16

SOFTWARE REQUIREMENTS SPECIFICATION

Online Voting System

University of Eswatini: Student Representative Council Elections

1.Introduction

This section provides an overview of the Software Requirements Specification (SRS) for an Online Voting System developed for the University of Eswatini (UNESWA). This document was prepared in accordance with IEEE Std 830-1998 and uses the functional hierarchy template (Annex A.7).

1.1 Purpose

This SRS defines the complete software requirements for The Online Voting System, a web-based online voting platform to be used for general student elections at the University of Eswatini. The document serves as the primary contractual and technical agreement between the student developer and the academic supervisor.

The intended audiences are:

- The student developer, as a binding specification to guide design, implementation, and testing.
- The academic supervisor, as an evaluation and review reference.
- The University of Eswatini student affairs administration, as a stakeholder reference for future deployment.
- Future developers or maintainers who may extend or modify the system.

1.2 Scope

The software product to be developed is an Online Voting System for the University of Eswatini Student Representative Council elections.

The Online Voting System is a web-based application that enables eligible registered students of UNESWA to cast votes in official student representative elections securely, anonymously, and remotely using any internet-connected device. The system will replace the current manual paper-ballot process used for student government elections.

The system will:

- Allow the Electoral Commission Administrator to create and manage elections, add candidates, and set voting windows.
- Authenticate voters using their UNESWA student number and a one-time password (OTP) delivered by email to their institutional address.
- Present eligible voters with a digital ballot and record their vote anonymously.
- Enforce a strict one-vote-per-voter rule for each election.
- Tabulate and display election results in real time upon closure of the voting window.
- Maintain a tamper-evident audit log for post-election review.

The system will NOT:

- Handle staff or faculty elections.
- Process financial transactions.

- Integrate with any external UNESWA student information system (SIS) in v1.0; voter data will be imported manually by the administrator.

Objectives and goals: The Online voting system aims to increase student voter participation by removing time barriers, improve result accuracy by eliminating manual counting, and strengthen trust in the electoral process through cryptographic vote integrity and comprehensive audit trails.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
UNESWA	University of Eswatini (formerly UNISWA)
UOE / UNESWA	Used interchangeably in this document to refer to the University of Eswatini
SRC	Student Representative Council — the governing body elected via online voting system
ECA	Electoral Commission Administrator — a designated university official managing elections
OTP	One-Time Password — a single-use code sent to a voter's institutional email for authentication
Voter	An enrolled UNESWA student eligible to participate in a given election
Ballot	The electronic form presented to a voter displaying candidates for a specific election
Election	A configured voting event with defined candidates, start time, and end time
Candidate	A student who has been approved by the ECA to stand in a specific election
Vote	A single anonymized selection made by an authenticated voter for one candidate
Audit Log	A tamper-evident chronological record of all system events
JWT	JSON Web Token — used for stateless session management
HTTPS	HyperText Transfer Protocol Secure — encrypted web communication protocol
WCAG	Web Content Accessibility Guidelines
API	Application Programming Interface
REST	Representational State Transfer — architectural style for web APIs
DB	Database
MVC	Model-View-Controller — software architectural pattern
OTP	One-Time Password
TBD	To Be Determined

1.4 References

Document / Standard	Details
IEEE Std 830-1998	IEEE Recommended Practice for Software Requirements Specifications. IEEE Computer Society, 1998.
IEEE Std 610.12-1990	IEEE Standard Glossary of Software Engineering Terminology.
OWASP Top Ten 2021	Open Web Application Security Project — Top Ten Web Application Security Risks. https://owasp.org
WCAG 2.1	Web Content Accessibility Guidelines (Level AA). W3C Recommendation, June 2018.
RFC 7519 (JWT)	JSON Web Tokens. Internet Engineering Task Force, May 2015.
GDPR / POPIA	General Data Protection Regulation (EU) / Protection of Personal Information Act (South Africa / Eswatini context) — data privacy reference.
UNESWA SRC Constitution	University of Eswatini Student Representative Council Electoral Rules (internal document).
Node.js Documentation	https://nodejs.org/en/docs
React.js Documentation	https://react.dev
MySQL 8.0 Reference Manual	https://dev.mysql.com/doc/refman/8.0/en/

1.5 Overview

The remainder of this SRS is structured as follows:

- Section 2: overall Description-provides background context including product perspective, summary of functions, user characteristics, general constraints, and assumptions.
- Section 3: Specific Requirements-contains all detailed and verifiable software requirements organized using the IEEE 830-1998 Annex A.7 Functional Hierarchy template. This section includes external interface requirements, functional requirements (information flows, process descriptions, data constructs, and a data dictionary), performance requirements, design constraints, and software system attributes.
- Appendix A: Glossary-provides extended term definitions.
- Appendix B: Use Case Summary-summarizes the primary use cases for traceability reference.

2. Overall Description

This section describes the general factors affecting the product and its requirements. It does not state specific requirements; rather, it provides context for the detailed requirements in Section 3.

2.1 Product Perspective

The Online voting system is a new, self-contained web-based application. It is not a component of a larger existing software system at UNESWA in this version. It replaces the current manual paper-ballot election process conducted by the Student Representative Council Electoral Commission.

The system will operate within the following environment:

- Client side: Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop, tablet, or smartphone.
- Server side: A cloud-hosted Node.js/Express web server with a MySQL relational database, deployed on Render.com.
- Network: The public internet (HTTPS). No special internal network is required; students may vote from any location.

The major system components and their relationships are described below:

Component	Description
Voter Portal (Frontend)	React.js single-page application (SPA) served over HTTPS. Provides login, ballot, and results interfaces.
Admin Panel (Frontend)	Separate React.js interface restricted to ECA users. Provides election and candidate management.
API Server (Backend)	Node.js + Express RESTful API handling authentication, vote logic, and data retrieval.
Database	MySQL relational database storing users, elections, candidates, votes, and audit logs.
Email Service	Third-party SMTP provider (e.g., SendGrid) used to deliver OTP codes to voters.
Cloud Host	Render.com (or equivalent), providing the server runtime and SSL certificate.

2.1.1 System Interfaces

The Online voting system interfaces with the UNESWA institutional email system for OTP delivery. Voter eligibility data is loaded into the system by the ECA using a CSV import function in the admin panel. No other system interfaces exist in v1.0.

2.1.2 User Interfaces

The system shall provide a browser-based graphical user interface. Key interface requirements include:

- Responsive design supporting screens from 320px (mobile) to 1920px (desktop) wide.
- Maximum 3 clicks from login to vote submission.
- Compliance with WCAG 2.1 Level AA accessibility guidelines.
- Support for English language only in v1.0.
- Clear visual indicators of election status (upcoming, active, and closed).

2.1.3 Hardware Interfaces

The system requires no specialized hardware. Voter client hardware requirements: any internet-connected device with a modern browser. Server hardware: managed by the cloud hosting provider. No dedicated hardware is procured for this project.

2.1.4 Software Interfaces

Software	Interface Description
Node.js v20 LTS	Runtime environment for the backend API server.
Express.js v4	Web framework for routing and middleware on the backend.
React.js v18	Frontend UI library for building the voter portal and admin panel.
MySQL 8.0	Relational database management system for persistent data storage.
SendGrid API v3	Third-party RESTful email API used to dispatch OTP messages.
bcrypt (npm)	Library for hashing and salting passwords before storage.
jsonwebtoken (npm)	Library for generating and verifying JWT session tokens.
Render.com	Cloud platform providing managed HTTPS hosting and SSL.

2.1.5 Communications Interfaces

All communication between the client browser and the API server shall use HTTPS (TLS 1.2 or higher). Unencrypted HTTP connections shall be automatically redirected to HTTPS. OTP delivery uses SMTP over TLS via the SendGrid API.

2.1.6 Memory Constraints

No specific memory constraints apply to client devices. The cloud server instance shall be configured with a minimum of 512 MB RAM. Database storage requirements are estimated at less than 1 GB for an election with up to 10,000 voters.

2.1.7 Operations

The system shall support the following operational modes:

- *Pre-Election Mode*: Election configuration, candidate management, and voter list import by the ECA. No votes may be cast.
- *Active Voting Mode*: Voting window is open. Authenticated voters may cast ballots. The admin panel remains accessible for monitoring. Results are not yet published.
- *Closed Mode*: Voting window has ended. Results are published. No new votes are accepted. Audit log is available to the ECA.
- *Maintenance Mode (administrator-initiated)*: System temporarily unavailable for updates. A maintenance notice page is displayed to all users.

2.1.8 Site Adaptation Requirements

The system shall be configurable per election instance. The ECA shall be able to set the following site-specific parameters via the admin panel without code changes:

- University name and logo displayed on the voter portal.
- Election title, description, and candidate list.
- Voting window start date/time and end date/time.
- Maximum votes per voter per election (default: 1).

2.2 Product Functions

The major functions of the Online Voting System will perform are summarized below.

Function Category	Summary
F1: Voter Authentication	Verify voter identity using UNESWA student number and OTP sent to institutional email. Establish a secure session.
F2: Election Management	Allow the ECA to create elections, add/remove candidates, upload voter lists, set voting windows, and publish results.
F3: Ballot Presentation	Display the appropriate ballot to an authenticated eligible voter. Enforce eligibility and one-vote-per-voter rules.
F4: Vote Casting	Record the voter's anonymous selection. Decouple voter identity from vote data. Prevent double voting.
F5: Vote Tabulation	Continuously count valid votes. Calculate percentage share per candidate.
F6: Results Publication	Display final results to all users (or ECA only, depending on configuration) upon election closure.
F7: Audit Logging	Record all significant system events with timestamps, user identifiers, and action descriptions to a tamper-evident log.
F8: Voter List Management	Allow the ECA to import, review, and manage the list of eligible voters per election via CSV upload.

2.3 User Characteristics

User Class	Characteristics
Voter (Student)	Enrolled UNESWA student. Variable technical proficiency — from first-year students with limited IT experience to postgraduate students. Expected to access the system on a smartphone. Should be able to complete the voting process without training beyond a brief on-screen guide. Proficient in English. No administrative or system knowledge assumed.
Electoral Commission Administrator (ECA)	A designated university official or IT staff member. Moderate to advanced technical proficiency. Responsible for configuring and managing elections. Will use a desktop or laptop computer. Will receive brief system training before each election cycle.
Academic Supervisor / Auditor	Reviews election results and audit logs post-election for quality assurance. Read-only access. No technical system knowledge required beyond basic web browsing.

2.4 Constraints

The following general constraints apply to the development and operation of Online Voting System:

- *Regulatory*: All voter personal data must be handled in accordance with the Protection of Personal Information Act (POPIA) principles applicable in the Kingdom of Eswatini context, including data minimization and consent.
- *Security*: The system must comply with the OWASP Top Ten 2021 security guidelines. Sensitive data (passwords, OTPs) must never be stored in plain text.
- *Ballot Anonymity vs. Authentication*: The system must simultaneously authenticate voters (to prevent fraud) and make the votes to be anonymous (to protect voter privacy). These are in fundamental tension and must be resolved architecturally by decoupling the voter identity record from the vote record at the point of submission.
- *Academic Context*: The system is developed as an undergraduate final-year project. The scope is limited to what a single developer can implement within 16 weeks. Advanced features (e.g., blockchain ledger, bio-metric authentication) are out of scope for v1.0.
- *Free-tier Infrastructure*: The system must operate within free-tier cloud hosting constraints.
- *Single Language*: The user interface shall support English only in v1.0.

2.5 Assumptions and Dependencies

The following assumptions are made. If any are invalid, requirements in Section 3 may need to be revised:

1. Voters have access to an active UNESWA institutional email address (@uneswa.ac.sz) and can receive OTP codes. If institutional email delivery is unreliable, an alternative OTP delivery channel (SMS) would need to be added.
2. The ECA will maintain an up-to-date CSV file of eligible voters (student ID, name, email) for each election and will import it before the voting window opens.
3. Voters have access to a device with an internet connection and a modern browser. If a significant portion of the student population lacks reliable internet access on campus, a campus-based voting kiosk solution may be needed.
4. The cloud hosting provider (Render.com) maintains at least 99% uptime during active voting windows.
5. Node.js v20 LTS and MySQL 8.0 remain available and supported throughout the project lifecycle.
6. The SendGrid free tier (100 emails/day) is sufficient for OTP delivery during a UNESWA election; if voter counts exceed 100 per day, a paid SendGrid plan is required.

3. Specific Requirements

This section contains all verifiable software requirements for Online Voting System. Requirements are organized using the IEEE 830-1998 Annex A.7 functional hierarchy template. Each requirement is uniquely identified and stated using the convention "The system shall..." Every requirement stated here must be met by the delivered software.

3.1 External Interface Requirements

3.1.1 User Interfaces

Req. ID	Requirement	Priority	Source
UI-01	Provide a public landing page displaying the system name, UNESWA branding, current election status (upcoming/active/closed), and a login button.	High	ECA/Student
UI-02	Display all pages with a responsive layout that renders correctly on-screen widths from 320px (mobile) to 1920px (desktop).	High	Student
UI-03	Present the voter login form with fields for Student Number and an OTP request button. The OTP entry field shall appear after the student number is validated.	High	Student
UI-04	Display the ballot as a list of candidate names with a radio-button selector and a single "Submit Vote" confirmation button. A confirmation dialog shall appear before final submission.	High	Student
UI-05	Display a "Vote Recorded Successfully" confirmation screen with a unique, non-reversible receipt reference code immediately after a successful vote submission.	High	Student
UI-06	Display election results as a bar chart and tabular breakdown (candidate name, votes received, percentage) after the election closes or when the ECA publishes results.	High	Student/ECA
UI-07	Provide the ECA admin panel with a navigation menu for: Elections, Candidates, Voters, Results, and Audit Log.	High	ECA
UI-08	Display all error messages in plain language without exposing internal system details, stack traces, or database error codes to the end user.	High	All
UI-09	Conform to WCAG 2.1 Level AA accessibility requirements, including sufficient colour contrast (minimum 4.5:1) and keyboard navigation support.	Medium	All
UI-10	Display session expiry warnings 2 minutes before the JWT session token expires and provide an option to extend the session.	Medium	Student

3.1.2 Hardware Interfaces

The Online Voting System does not interface directly with any hardware devices. All hardware interaction is managed by the operating system and browser of the client device or the cloud server infrastructure provided by Render.com. No hardware interface specifications beyond those implied by HTTPS/TCP-IP networking are applicable.

3.1.3 Software Interfaces

Req. ID	Requirement	Priority	Source
SI-01	Interface with MySQL 8.0 using the mysql2 npm driver. All database queries shall use parameterized statements to prevent SQL injection.	High	Developer
SI-02	Interface with the SendGrid Email API v3 via HTTPS POST requests to dispatch OTP emails. The system shall handle SendGrid API errors gracefully and log them without exposing credentials.	High	Developer
SI-03	Issue signed JSON Web Tokens (JWT, RS256 algorithm) upon successful authentication. All protected API routes shall validate the JWT on every request.	High	Developer
SI-04	Serve all frontend assets via a CDN-compatible static file structure to reduce latency for students accessing the system from off-campus.	Medium	Developer

3.1.4 Communications Interfaces

Req. ID	Requirement	Priority	Source
CI-01	enforce HTTPS (TLS 1.2 or higher) for all client-server communication. All HTTP requests shall be permanently redirected (HTTP 301) to HTTPS.	High	Developer
CI-02	use RESTful HTTP/1.1 API conventions. All API responses shall be in JSON format with appropriate HTTP status codes (200, 201, 400, 401, 403, 404, 409, 500).	High	Developer
CI-03	implement CORS (Cross-Origin Resource Sharing) headers to restrict API access to the designated frontend domain only.	High	Developer
CI-04	transmit OTP emails via SMTP over TLS using the SendGrid relay. OTP codes shall expire within 10 minutes of issuance.	High	Developer

3.2 Functions

This section is organized using the IEEE 830-1998 Annex A.7

Req. ID	Requirement	Priority	Source
FR-01	accept a valid UNESWA student number (7-digit integer string) and check it against the registered voter list for the active election.	High	ECA/Student
FR-02	generate a 6-digit cryptographically random OTP and send it to the voter's registered institutional email address within 5 seconds of request.	High	Student

FR-03	invalidate an OTP after a single successful use or after its 10-minute expiry period, whichever comes first.	High	Student
FR-04	lock a voter account for 15 minutes after 5 consecutive failed OTP attempts for the same student number and election.	High	Security
FR-05	issue a signed JWT with a 30-minute expiry upon successful OTP verification, encoding the voter ID, election ID, and user role.	High	Developer
FR-06	reject all API requests to protected routes that do not carry a valid, unexpired JWT.	High	Security

3.3 Performance Requirements

The following measurable performance requirements apply to the Online Voting System:

Req. ID	Requirement	Priority	Source
PR-01	respond to 95% of voter authentication API requests within 3 seconds under normal load (up to 200 concurrent users).	High	Student
PR-02	respond to 95% of ballot retrieval requests within 2 seconds under normal load.	High	Student
PR-03	respond to 95% of vote submission requests within 3 seconds under normal load.	High	Student
PR-04	support a minimum of 200 concurrent authenticated voters without degradation of response time targets stated in PR-01 through PR-03.	High	ECA
PR-05	complete vote tabulation and make results available within 60 seconds of election closure, for elections with up to 10,000 voters.	High	ECA
PR-06	maintain a database capable of storing at least 10,000 voter records and 50,000 vote records without performance degradation.	Medium	ECA
PR-07	deliver OTP emails to voters within 30 seconds of the OTP generation request, subject to email provider service levels.	High	Student

3.4 Design Constraints

Req. ID	Requirement	Priority	Source
DC-01	be implemented as a web application using Node.js (v20 LTS) and Express.js (v4) for the backend API.	High	Developer
DC-02	be implemented using React.js (v18) for the frontend user interface.	High	Developer
DC-03	use MySQL 8.0 as the relational database management system.	High	Developer

DC-04	follow the MVC (Model-View-Controller) architectural pattern for the backend codebase.	High	Developer
DC-05	use bcrypt with a minimum work factor of 12 for all password and OTP hashing operations.	High	Security
DC-06	comply with the OWASP Top Ten 2021 security guidelines. Specifically: protection against SQL Injection (A03), Broken Authentication (A07), Security Misconfiguration (A05), and Cryptographic Failures (A02).	High	Security
DC-07	be deployable on the Render.com free-tier cloud platform without requiring custom server configuration beyond environment variables.	High	Developer
DC-08	manage all source code using Git version control with a public GitHub repository. Commits shall be made at minimum after each completed development phase.	High	Developer
DC-09	include a comprehensive README.md file documenting installation, configuration, and deployment steps.	Medium	Developer

3.5 Software System Attributes

3.5.1 Reliability

Req. ID	Requirement (The system shall...)	Priority	Source
RA-01	Achieve a minimum uptime of 99% during any active election voting window, as measured over the duration of that window.	High	ECA
RA-02	Implement database transactions for all vote write operations so that partial failures leave the database in a consistent state (no half-recorded votes).	High	Developer
RA-03	Implement automatic retry logic (maximum 3 retries with exponential backoff) for failed OTP email delivery attempts.	Medium	Developer
RA-04	Log all unhandled server errors to a persistent error log and return a generic HTTP 500 response to the client without exposing internal details.	High	Developer

3.5.2 Availability

Req. ID	Requirement	Priority	Source
AV-01	Be available 24 hours a day, 7 days a week during the period between the configured elections start and end date.	High	ECA/Student
AV-02	Display a scheduled maintenance notice page during planned downtime, configured by the ECA at least 24 hours in advance.	Medium	ECA
AV-03	Support horizontal scaling by externalizing all session state to JWT tokens (stateless API), ensuring no server-side session storage is required.	Medium	Developer

3.5.3 Security

Req. ID	Requirement	Priority	Source
SE-01	Store all voter passwords and OTP codes as bcrypt hashes (work factor ≥ 12). Plain-text credentials shall never be written to any persistent storage, log, or API response.	High	Security
SE-02	Implement rate limiting on the OTP request endpoint: maximum 3 OTP requests per student number per 15-minute window.	High	Security
SE-03	Implement rate limiting on all authentication endpoints: maximum 10 requests per IP address per minute.	High	Security
SE-04	Set all session cookies (if used) with the Http only, Secure, and Same Site=Strict flags.	High	Security
SE-05	Implement CSRF protection on all state-changing API endpoints.	High	Security
SE-06	Validate and sanitize all user inputs on both the client and server side before processing.	High	Security
SE-07	Ensure that the Votes table does not contain any column that references or can be joined to identify a specific voter's choice, preserving ballot anonymity by design.	High	Privacy
SE-08	Enforce role-based access control: voter-role JWTs shall have no access to admin panel endpoints. Admin-role JWTs shall not be usable to cast a vote.	High	Security
SE-09	Include Content Security Policy (CSP), X-Frame-Options: DENY, and X-Content-Type-Options: no sniff HTTP headers in all API responses.	Medium	Security

3.5.4 Maintainability

Req. ID	Requirement	Priority	Source
MT-01	Achieve a minimum test coverage of 70% for backend API logic, measured using Jest code coverage reports.	Medium	Developer
MT-02	Organize backend code into clearly named modules: routes/, controllers/, models/, middleware/, and utils/.	Medium	Developer
MT-03	Include inline code comments for all non-trivial algorithms, particularly the vote anonymity, OTP generation, and database transaction logic.	Medium	Developer
MT-04	Use environment variables (via a .env file and the dotenv library) for all configurable parameters including database credentials, JWT secret, and SMTP API key. These values shall never be hardcoded in source files.	High	Developer

3.5.5 Portability

Req. ID	Requirement	Priority	Source
PO-01	Be deployable on any cloud platform supporting Node.js v20 LTS and MySQL 8.0, including but not limited to Render.com, Railway, and Heroku, with no code changes — only environment variable updates.	Medium	Developer
PO-02	Use only npm packages that are compatible with Linux, macOS, and Windows development environments.	Medium	Developer
PO-03	Provide a Docker Compose file (docker-compose.yml) for local development environment setup as part of the project repository.	Low	Developer

3.6 Other Requirements

Req. ID	Requirement	Priority	Source
OR-01	Provide a voter help page accessible without authentication, explaining the voting process in plain English with numbered steps.	Medium	Student
OR-02	Display all datetimes to end users in Eswatini Standard Time (UTC+2), while storing all datetimes in the database as UTC.	High	Student/ECA
OR-03	send a confirmation email to each voter after their vote is successfully recorded. The email shall include the receipt token but not the candidate choice.	Medium	Student
OR-04	allow the ECA to reset a voter has voted flag only with a documented justification stored in the audit log, to handle cases of verified technical failure.	Low	ECA

Appendix A: Extended Glossary

The following terms supplement the definitions provided in Section 1.3.

Term	Extended Definition
Atomic Transaction	A database operation that is treated as a single indivisible unit of work: either all steps succeed (commit) or all steps are undone (rollback). Used in Online voting system to ensure vote recording and has voted flag updates are always consistent.
Ballot Anonymity	The property that, after a vote is cast, it is computationally infeasible to determine which voter cast any particular vote. Achieved in the Online Voting system by architectural separation of the Votes table from any voter identity.
bcrypt	A password hashing function designed to be computationally expensive, making brute-force attacks impractical. The work factor (cost) determines computational effort required per hash.
Cold Start Latency	Delay experienced when a cloud server instance has been idle and must restart before handling a request. Relevant to Render.com free-tier deployments.
CORS	Cross-Origin Resource Sharing — a browser security mechanism that restricts web pages from making requests to a different domain than the one that served the page.
CSRF	Cross-Site Request Forgery — an attack that tricks a victim into submitting a malicious request using their authenticated session.
Electoral Commission Administrator (ECA)	The designated university official responsible for managing the Online Voting System system during an election cycle. Holds the admin role within the system.
Functional Hierarchy	An IEEE 830-1998 Annex A.7 approach to organize software requirements by decomposing functionality into a tree of data flows, processes, and data constructs.
Has-Voted Flag	A Boolean field in the Voter_Election table that is set to TRUE when a voter successfully casts their ballot, preventing double voting.
JWT (JSON Web Token)	A compact, cryptographically signed token used for stateless authentication. The server issues a JWT upon login; the client presents it with every subsequent request.
OTP (One-Time Password)	A 6-digit numeric code generated randomly for each authentication attempt, valid for 10 minutes and single-use.
Rate Limiting	A security control that restricts the number of requests a user or IP address can make within a defined time window, mitigating brute-force and denial-of-service attacks.

Receipt Token	A UUID v4 string returned to a voter after successful vote submission, providing non-repudiable proof of participation without revealing candidate choice.
SRC (Student Representative Council)	The elected student government body at UNESWA. Online Voting System is designed to conduct SRC general elections.
UUID v4	Universally Unique Identifier version 4 — a randomly generated 128-bit value formatted as xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx, with negligible collision probability.

Appendix B: Use Case Summary

The following table provides a traceability reference mapping primary use cases to their corresponding requirements in Section 3. This summary does not replace the detailed requirements but aids in testing and verification planning.

Use Case ID	Use Case Name	Actor	Related Requirements
UC-01	Voter Login with OTP	Student Voter	FR-01, FR-02, FR-03, FR-04, FR-05, FR-06, SE-01, SE-02, SE-03
UC-02	View and Cast Ballot	Student Voter	FR-15, FR-16, FR-17, FR-18, FR-19, FR-20, FR-21, FR-22, FR-23, UI-04, UI-05
UC-03	View Election Results	Student / Public	FR-27, FR-28, FR-29, UI-06
UC-04	Create Election	ECA	FR-07, FR-08, UI-07
UC-05	Add / Remove Candidates	ECA	FR-11, UI-07
UC-06	Import Voter List (CSV)	ECA	FR-12, FR-13
UC-07	Open / Close Election	ECA (automated)	FR-09, FR-10
UC-08	Publish Election Results	ECA	FR-30, FR-24, FR-25, FR-26
UC-09	View Audit Log / Export	ECA / Auditor	FR-31, FR-32, FR-33, FR-34
UC-10	Monitor Voter Turnout	ECA	FR-14